

Отлично, я подготовлю подробный гайд по паттернам программирования в PHP 8+ с универсальными примерами, но с пояснениями по специфике Laravel и Symfony (например, отличие фабрик и фасадов в Laravel от одноименных паттернов). Будут рассмотрены как классические паттерны (GoF, GRASP), так и современные подходы (DDD, CQRS, Event Sourcing и др.).

В гайде будут:

- Практические примеры с разбором плюсов и минусов
- Влияние паттернов на тестируемость кода
- Анализ влияния паттернов на производительность
- Антипаттерны с альтернативными решениями
- Обзор популярных библиотек, реализующих паттерны
- Пояснения сложных концепций на примерах из реальной жизни
- Шпаргалка-выжимка для быстрого использования

Я сообщу, когда материал будет готов!

Паттерны программирования в PHP 8+: подробный гайд для Senior Backend- разработчиков

1. Универсальные примеры паттернов и специфика Laravel/Symfony

Паттерны проектирования – это шаблоны решений общих архитектурных задач. В PHP 8+ они применяются как напрямую, так и опосредованно через фреймворки Laravel и Symfony. Важно понимать, как классические паттерны реализуются во фреймворках и чем их реализации отличаются от одноименных шаблонов проектирования.

- **Фасады (Facades)**. В GoF-паттерне *Facade* фасад предоставляет упрощенный интерфейс к сложной подсистеме классов ([Does Facade in Laravel implements Facade or Proxy Pattern? - Stack Overflow](#)) ([Does Facade in Laravel implements Facade or Proxy Pattern? - Stack Overflow](#)). В Laravel термин *Facade* означает нечто слегка иное: это *статические прокси* для объектов из IoC-контейнера ([Facades - Laravel 11.x - The PHP Framework For Web Artisans](#)). Например, вызов `Cache::get()` в Laravel фактически перенаправляется через фасад к объекту кеша в контейнере ([Facades - Laravel 11.x - The PHP Framework For Web Artisans](#)). Такой Laravel-фасад скрывает сложность доступа к объекту и обеспечивает лаконичный синтаксис. В отличие от классического шаблона, Laravel-фасад обычно обортывает **один** объект (а не подсистему), поэтому ближе к паттерну *Proxy* ([Does Facade in Laravel implements Facade or Proxy Pattern? - Stack Overflow](#)) ([Does Facade in Laravel implements Facade or Proxy Pattern? - Stack Overflow](#)). Тем не менее, Laravel-фасады всё же реализуют идею фасада – упрощают интерфейс и скрывают реализацию сервиса ([Does Facade in Laravel implements Facade or Proxy Pattern? - Stack Overflow](#)) ([Does Facade in Laravel implements Facade or Proxy Pattern? - Stack Overflow](#)). Symfony напрямую не использует "Facades" как Laravel, вместо этого предпочитая явное внедрение зависимостей (Dependency Injection) и сервисы.

- **Фабрики (Factories).** Классический *Factory Method* или *Abstract Factory* управляют созданием объектов, скрывая детали инстанцирования ([Design Patterns: Factory vs Factory method vs Abstract Factory](#)). В Laravel и Symfony термин "фабрика" тоже встречается, но в другом контексте. **Laravel Model Factory** – это средство для генерации тестовых данных: классы-фабрики создают заполненные экземпляры моделей для сидирования БД и тестов ([Eloquent: Factories - Laravel 11.x - The PHP Framework For Web ...](#)). Такая "фабрика" – скорее утилита фреймворка, а не паттерн проектирования. В Symfony можно определять фабричные сервисы в контейнере (методы `factory` в YAML/XML конфигурации) – по сути это реализация шаблона *Factory*: контейнер вызывает указанный метод, чтобы сконструировать объект сервиса. Разница в том, что Laravel-фабрики и фасады – понятия, специфичные для фреймворка, тогда как шаблоны *Factory* и *Facade* – общезыковые паттерны. Необходимо различать: **фасад Laravel** (статический прокси) и **паттерн Facade**, **фабрики Laravel** (для данных) и **паттерн Factory** (для абстракции создания объектов).
- **Active Record vs Data Mapper.** Ещё один пример различий – паттерны доступа к данным в ORM. Laravel Eloquent реализует паттерн *Active Record* – каждая модель отвечает за взаимодействие со своей таблицей (CRUD-методы прямо в классе модели) ([My Digital Garden | Active Record vs. Data Mapper: Navigating PHP ORM Patterns with Eloquent and Doctrine](#)) ([My Digital Garden | Active Record vs. Data Mapper: Navigating PHP ORM Patterns with Eloquent and Doctrine](#)). Symfony обычно использует Doctrine, которая построена на паттерне *Data Mapper*: сущности (Entity) не знают о БД, а отдельный компонент (EntityManager + Unit of Work) занимается загрузкой/сохранением данных ([My Digital Garden | Active Record vs. Data Mapper: Navigating PHP ORM Patterns with Eloquent and Doctrine](#)) ([My Digital Garden | Active Record vs. Data Mapper: Navigating PHP ORM Patterns with Eloquent and Doctrine](#)). **Active Record** проще и быстрее начать, модель содержит и данные, и методы БД, например `User::find(1)` загрузит пользователя ([My Digital Garden | Active Record vs. Data Mapper: Navigating PHP ORM Patterns with Eloquent and Doctrine](#)). **Data Mapper** разделяет логику: объект-сущность чисто хранит состояние, а persistence-слой управляет БД ([My Digital Garden | Active Record vs. Data Mapper: Navigating PHP ORM Patterns with Eloquent and Doctrine](#)) ([My Digital Garden | Active Record vs. Data Mapper: Navigating PHP ORM Patterns with Eloquent and Doctrine](#)). В итоге Eloquent-модели тесно связаны с БД (проще в небольших проектах), а Doctrine-сущности легко тестировать и менять, так как бизнес-логика отделена от SQL. При росте сложности Data Mapper обычно обеспечивает лучшую масштабируемость архитектуры ценой более сложной настройки.
- **Другие спецификации.** Laravel и Symfony применяют и другие паттерны: например, *Front Controller* – единая точка входа (`public/index.php`), которая делегирует обработку запроса фреймворку. Оба используют *Dependency Injection* (DI) – контейнер сервисов: в Symfony контроллеры получают зависимости через конструктор, Laravel позволяет то же, но также имеет фасады и *Service Locator*-подход через `App::make()` или глобальные помощники. Symfony строго придерживается DI, Laravel более гибок, предлагая синтаксический сахар, который потенциально скрывает зависимости. Понимание этих отличий позволяет правильно применять шаблоны: например, Laravel-фасады удобны, но из-за их статической природы нужно осознавать, что за ними стоит контейнер (для тестирования можно подменить binding). Symfony же явно требует DI, что поощряет хороший дизайн с явными зависимостями.

2. Классические паттерны (GoF, GRASP) и современные подходы (DDD, CQRS, Event Sourcing)

Разработчики используют **классические паттерны проектирования** (формализованные «бандой четырёх», GoF) и **принципы GRASP** для построения гибкой архитектуры. Со временем появились и **современные подходы** на более высоком уровне (DDD, CQRS, Event Sourcing и др.), которые решают масштабные задачи проектирования приложений. Рассмотрим и те, и другие.

Паттерны GoF (Gang of Four)

GoF-паттерны – 23 классических шаблона, разделённых на три группы: порождающие, структурные и поведенческие. Ниже кратко описаны некоторые из них и примеры их использования в PHP.

- **Порождающие (Creational):** помогают гибко создавать объекты.
 - *Factory Method* (Фабричный метод) – определяет интерфейс для создания объекта, но позволяет подклассам решать, какой класс инстанцировать ([Design Patterns: Factory vs Factory method vs Abstract Factory](#)). Например, в зависимости от настройки приложения можно через фабричный метод выбирать, какой класс логгера создать (FileLogger или DbLogger).
 - *Abstract Factory* (Абстрактная фабрика) – предоставляет интерфейс для создания нескольких взаимосвязанных объектов без привязки к конкретным классам ([Design Patterns: Factory vs Factory method vs Abstract Factory](#)). В PHP примером может быть фабрика GUI-компонентов, которая в зависимости от ОС создаёт набор классов (кнопка, чекбокс) под Windows или под macOS.
 - *Singleton* (Одиночка) – гарантирует единственный экземпляр класса и глобальную точку доступа к нему. Например, в старом коде часто делали синглтон для объекта базы данных. Однако Singleton нередко считается **антипаттерном** из-за скрытой глобальности и проблем с тестированием ([Why is Singleton considered an anti-pattern? - Stack Overflow](#)) ([Why is Singleton considered an anti-pattern? - Stack Overflow](#)).
 - *Builder* (Строитель) – отделяет конструирование сложного объекта от его представления, позволяя использовать один и тот же процесс строительства для разных представлений. В веб-разработке отличный пример – **Query Builder** для SQL. В Laravel Query Builder реализует шаблон Builder: методами вроде `->where()->orderBy()->get()` пошагово строится объект запроса ([08 - Builder Pattern: You're Already Using It Daily | Laravel Daily](#)) ([08 - Builder Pattern: You're Already Using It Daily | Laravel Daily](#)).
 - *Prototype* (Прототип) – позволяет копировать существующие объекты, не зависимо от их конкретных классов, через клонирование. Может применяться для дублирования сложных объектов (в PHP можно использовать `clone`).
- **Структурные (Structural):** определяют способы построения отношений между объектами.
 - *Adapter* (Адаптер) – преобразует интерфейс одного класса к виду, совместимому с ожиданиями другого. Например, класс-адаптер может позволить работать с разными API хранения (Memcache, Redis) через единый интерфейс CacheInterface.

- *Decorator* (Декоратор) – динамически добавляет объекту новые обязанности, оборачивая его. В PHP так декорируют, например, поток вывода или логгер: объект-декоратор добавляет функциональность (например, форматирование, буферизацию) к оригинальному объекту, не меняя его интерфейс.
 - *Facade* (Фасад) – предоставляет унифицированный интерфейс к подсистеме. Как упоминалось, Laravel-фасады упрощают доступ к компонентам фреймворка. Классический пример фасада – класс, который инкапсулирует взаимодействие с несколькими сложными классами (например, фасад "Домашний кинотеатр" с методами `watchMovie()` может внутри запускать проектор, опускать экран, настраивать звук и т.д.).
 - *Proxy* (Прокси) – предоставляет суррогат для реального объекта, контролируя доступ к нему. Например, *Lazy Proxy*: объект-заглушка, откладывающий создание тяжёлого ресурса до момента первого обращения. Symfony использует прокси для ленивой загрузки сервисов, Doctrine – для ленивой загрузки связанных сущностей.
 - *Composite* (Компоновщик) – организует объекты в древовидную структуру для представления иерархий "часть-целое", позволяя клиенту единообразно работать и с отдельными, и с составными объектами. Пример – построение HTML-дерева элементов: листовые узлы – конкретные виджеты, узлы-компоновщики – группы элементов, при этом каждый реализует общий интерфейс рендеринга.
 - *Bridge* (Мост) – разделяет абстракцию и реализацию, позволяя изменять их независимо. Например, класс отображения графика может использовать "мост" к разным библиотекам рисования (GD, Imagick) через общий интерфейс.
 - *Flyweight* (Приспособленец) – позволяет много мелких объектов хранить эффективно, разделяя общий состояния. Пример – кэширование уже созданных объектов, чтобы не дублировать их в памяти (например, объекты символов в текстовом редакторе).
- **Поведенческие (Behavioral):** определяют способы эффективного взаимодействия между объектами, распределения обязанностей.
- *Observer* (Наблюдатель) – устанавливает зависимость типа "издатель-подписчик": объекты-наблюдатели подписываются на события некоего субъекта и оповещаются при изменениях ([What is the difference between Repository Pattern and Facades ...](#)). В PHP это реализовано в системе событий: например, в Symfony EventDispatcher позволяет одному компоненту отправить событие, а другим – среагировать (слушатели). Пример из жизни: издатель новостей уведомляет подписчиков, когда выходит новая статья.
 - *Strategy* (Стратегия) – определяет семейство алгоритмов, инкапсулирует их и делает взаимозаменяемыми для клиента. Например, стратегия расчёта стоимости доставки: можно подставить алгоритм курьерской доставки или почтой, в зависимости от условий, не меняя код заказа. Это избавляет от громоздких `if/else` и облегчает расширение (добавление новых алгоритмов) ([Design Patterns in PHP: Facade \(with Laravel\) - DEV Community](#)).
 - *Command* (Команда) – инкапсулирует запрос в виде объекта, позволяя параметризовать и хранить операции, отменять их (`undo/redo`). В PHP команды используются в CQRS (каждая команда – действие, которое выполняется), а также в очередях (`jobs`).
 - *Chain of Responsibility* (Цепочка обязанностей) – передаёт запрос по цепочке обработчиков, пока кто-то не обработает. Яркий пример – **HTTP Middleware**. В Laravel и Symfony цепочка `middlewares` последовательно

применяет обработчики к запросу: каждый может либо полностью обработать его, либо вызвать следующий. Этот шаблон позволяет разделить обработку (аутентификация, логирование, кэширование) на отдельные звенья.

- *Template Method* (Шаблонный метод) – определяет скелет алгоритма в базовом классе, оставляя реализацию некоторых шагов подклассам. Например, абстрактный класс экспорта `Exporter` может иметь метод `export()` с общей логикой (открыть файл, записать заголовок, **hook** для основного содержимого, закрыть файл). Подклассы (`CsvExporter`, `PdfExporter`) переопределяют только часть – генерацию содержимого.
- *State* (Состояние) – объект изменяет своё поведение при смене внутреннего состояния, как будто меняет класс. Применим для объектов, сильно зависящих от состояния. Пример: заказ Интернет-магазина может быть в состояниях "Новый", "Оплачен", "Отправлен", "Доставлен" – с разным поведением методов в каждом состоянии. Реализуется выделением отдельных классов для каждого состояния, которые реализуют общий интерфейс действий заказа.
- *Mediator* (Посредник) – определяет объект, инкапсулирующий взаимодействие множества других объектов, устраняя "все-ко-всем" связи. Например, объект `ChatRoom` выступает посредником между множеством объектов Пользователь – пользователи отправляют сообщения через комнату, которая решает, кому пересылать. В Symfony посредником можно считать шину сообщений (`Messenger Bus`), через которую разные компоненты обмениваются задачами.
- *Memento* (Хранитель) – сохраняет и восстанавливает состояние объекта без нарушения инкапсуляции. В PHP например можно сохранять состояние объекта формы, чтобы потом откатить (не часто применимо вне специфичных задач).
- *Visitor* (Посетитель) – отделяет операцию от структуры объектов, позволяя добавлять новые операции, не меняя классы. Например, есть разные типы AST-узлов (в парсере) и нужно выполнять разные действия – шаблон `Visitor` позволяет создать класс посетителя с методами для каждого типа узла (интерпретация, компиляция и т.п.), а сами узлы принимают посетителя.

Выше перечислены наиболее распространённые GoF-паттерны. Многие из них на практике присутствуют в коде Laravel/Symfony: напрямую или в сочетаниях. Например, **PSR-11 контейнер** – это реализация *Service Locator* (который часто критикуется, см. антипаттерны), **PSR-14 events** – реализация *Observer*, **PSR-15 middleware** – *Chain of Responsibility*. Понимание этих шаблонов помогает как при чтении внутренностей фреймворков, так и при проектировании собственной архитектуры.

Принципы GRASP

Помимо GoF, известны и принципы **GRASP** (General Responsibility Assignment Software Principles) – набор из 9 фундаментальных принципов объектно-ориентированного дизайна. Они помогают решить, **каким классам** давать ту или иную ответственность. Кратко эти принципы:

- **Информационный эксперт** – ответственность возлагается на тот класс, который обладает необходимой информацией для выполнения задачи.
- **Создатель (Creator)** – определяет, какой объект должен создавать другие: класс А создаёт класс В, если А содержит или агрегирует В, либо А использует В.
- **Контроллер** – отдельно выделенный объект принимающий запросы от UI или внешних событий и направляющий их обработку (например, *контроллеры MVC* следуют этому принципу).

- **Слабое зацепление (Low Coupling)** – минимизация связностей между модулями для облегчения модификации и повторного использования.
- **Высокая Cohesion (High Cohesion)** – класс должен быть сфокусирован на одной задаче (сродни принципу единственной ответственности из SOLID), что повышает понятность и поддержку.
- **Полиморфизм** – использование полиморфных вызовов (интерфейсов/абстракций) вместо условных операторов для выбора поведения. Например, вместо `if` по типу документа – иметь классы-наследники и вызывать метод.
- **Допустимая сложность (Protected Variations)** – защищать от влияния изменяющихся требований, вводя абстракции или антиципируя изменения (принцип «закрытости» – OCP).
- **Посредник (Indirection)** – ввод промежуточных объектов для уменьшения прямой связи между двумя компонентами (например, тот же Mediator паттерн).
- **Чистая выдумка (Pure Fabrication)** – когда нет естественного класса-эксперта, придумать искусственный класс для ответственности (например, сервисы, хелперы, утилиты – чтобы не перегружать бизнес-объекты лишним).

Применение GRASP-принципов помогает спроектировать классы до того, как вставлять готовые паттерны. Например, *Information Expert* и *Low Coupling* подсказывают использование паттерна *Repository*: выделить слой доступа к данным, чтоб бизнес-логика не знала деталей БД. *Controller* приводит к MVC разделению в фреймворках. Хотя GRASP не дают конкретного кода, они обеспечивают философию распределения обязанностей, лежащую в основе грамотного использования паттернов.

Domain-Driven Design (DDD)

Domain-Driven Design (Предметно-ориентированное проектирование) – современный подход к разработке сложных программ, ставящий во главу **бизнес-домен**. DDD – это не один паттерн, а совокупность стратегических и тактических паттернов и принципов, помогающих выстраивать софт вокруг предметной области.

Основные понятия DDD (тактические паттерны DDD) применимы и в PHP:

- **Сущность (Entity)** – объект домена, имеющий уникальную идентичность и жизненный цикл ([Domain-Driven Design with PHP | The PHP Consulting Company](#)). Например, *User*, *Order* в системе – сущности, у которых есть идентификатор. Они изменяемы и их равенство определяется ID, а не полями.
- **Value Object (Объект-значение)** – неизменяемый объект без уникального идентификатора, равенство по значению. Пример: *Money* (валюта и сумма) или *Address*. Value Object лучше делает модель более выразительной: вместо нескольких примитивов – один логический объект. В Laravel Value Objects можно использовать для типизации полей VO (например, кастомные типы Eloquent casting) или просто как отдельные классы.
- **Aggregate (Агрегат)** – группа связанных объектов (сущностей и VO), объединённых вокруг одной корневой сущности (Aggregate Root). Агрегат обеспечивает целостность инвариантов: внешнему миру доступен только корень. Например, *Order* и его *OrderItems*: *Order* – корень агрегата, операции с заказом (добавить товар, подтвердить) проходят через него, а не напрямую к элементам, чтобы гарантировать бизнес-правила.
- **Repository (Репозиторий)** – паттерн для хранения и извлечения агрегатов из источника данных, имитируя коллекцию объектов ([Domain-Driven Design with PHP | The PHP Consulting Company](#)) ([Domain-Driven Design with PHP | The PHP Consulting Company](#)). В DDD репозиторий – это *interface*, скрывающий детали БД, и часть слоя домена (а не инфраструктуры). Например, *OrderRepository* с методами `save(Order`

`$order)` и `findById(OrderId $id)`. В Symfony Doctrine обычно предоставляет *EntityRepository*, но с точки зрения DDD его можно обернуть своим интерфейсом. Репозиторий упрощает тестирование (можно замокать интерфейс) ([Domain-Driven Design with PHP | The PHP Consulting Company](#)) ([Domain-Driven Design with PHP | The PHP Consulting Company](#)) и воспринимается как коллекция агрегатов.

- **Factory (Фабрика домена)** – в DDD это фабрика для создания сложных объектов (агрегатов) с соблюдением инвариантов. Если создание объекта требует сложной логики (например, создать заказ сразу с позициями, проверив остатки), выносят эту логику в доменную фабрику (статический метод или отдельный класс).
- **Domain Event (Событие домена)** – событие, отражающее важный факт в домене, например "PaymentReceived" (платёж получен) или "OrderShipped". Domain Events используются для уведомления других частей системы о произошедшем. В PHP их можно реализовать простым классом-событием и диспетчером (напрямую или через шину сообщений).

DDD также включает **стратегические паттерны**: *Bounded Context* (ограниченный контекст, обособленный раздел домена со своей моделью), *Ubiquitous Language* (единый язык для разработчиков и экспертов предметной области) и т.д. Эти аспекты влияют на архитектуру приложения в целом. Например, большие приложения на Laravel часто разделяют на контексты (модули), каждый со своими моделями и логикой, чтобы избежать "Большого кома грязи" (Big Ball of Mud).

Стоит отметить, что DDD не заменяет GoF-паттерны, а строится поверх них. Реализуя DDD-модель в PHP, вы всё равно используете фабрики, репозитории, наблюдателей (события) – просто применяете их осмысленно в контексте предметной области. Например, **репозиторий** в DDD – частный случай паттерна *Facade* (он скрывает детали persistence, представляя простой интерфейс домену) ([Domain-Driven Design with PHP | The PHP Consulting Company](#)) ([Domain-Driven Design with PHP | The PHP Consulting Company](#)). DDD особенно полезно при сложных бизнес-правилах, помогая держать код гибким к изменениям в логике, а не только к изменениям технических требований.

CQRS

CQRS (Command Query Responsibility Segregation) – разделение операций чтения и записи. Идея: вместо одной модели данных/объекта, который и читает, и пишет, мы используем **раздельные модели**: командную (для изменений) и запросную (для чтения). Команды (Command) изменяют состояние, но ничего не возвращают, а запросы (Query) только читают данные, но не меняют.

Практическое применение CQRS в PHP обычно выглядит так:

- **Write-модель** (командная модель) – представленная, например, набором *Command*-объектов (DTO с данными команды) и *Command Handler*-ов (обработчиков, применяющих команды). Команда могла бы быть `RegisterUserCommand { name, email, ... }`, обработчик – `RegisterUserHandler`, который создает новую сущность `User`, валидирует, сохраняет через репозиторий.
- **Read-модель** – отдельный механизм получения данных, оптимизированный для запросов. Это может быть упрощённый набор DTO/Entities для отображения, или даже отдельная база данных/индекс. Например, после регистрации пользователя write-модель сохранила `User` в нормализованной БД, а read-модель может обращаться к поисковому индексу или кэшированному представлению пользователей для отображения на сайте.

CQRS часто сопровождается **Event Sourcing** (о нём далее) и использованием *Eventual Consistency* (запись и чтение могут быть несинхронны: команда записывает событие, а

read-модель обновляется немного позже). Но CQRS можно применять и без Event Sourcing: например, в Symfony Messenger можно реализовать команды (как сообщения) и отдельные запросы к базе.

В Laravel/Symfony можно организовать CQRS так:

- Использовать библиотеку **Tactician** или аналог для диспетчеризации команд (т.е. реализация паттерна *Command Bus*).
- Для запросов – либо прямой доступ к БД/репозиторию, либо отдельный слой "Query Services/Query Handlers".

Плюсы CQRS: читательская часть можно оптимизировать независимо (например, денормализовать данные для быстрого отображения списков), а записывающая – сосредоточиться на бизнес-логике. Упрощается масштабирование: можно вынести чтение в реплики БД или другой хранилище. **Минусы:** усложнение архитектуры, дублирование кодовых сущностей (отдельные классы для команд и для запросов). Применять CQRS стоит в случаях, когда нагрузка чтения и записи очень различается или сложная доменная логика требует четкого отделения процесса изменения состояния.

Event Sourcing

Event Sourcing (ES) – способ хранения состояния системы через последовательность событий, а не через хранение только текущего состояния. При Event Sourcing **каждое событие изменения** (например, "BalanceCredited", "BalanceDebited" в банковской системе) записывается в хранилище событий (event store). Текущее состояние агрегата не сохраняется напрямую – оно получается *проигрыванием* (replaying) всех событий, относящихся к этому агрегату.

Как это выглядит:

- У каждого агрегата (например, банковский счёт) есть список событий. Новое событие добавляется при каждом изменении (пополнение, списание).
- Чтобы получить текущее состояние (баланс), система загружает все события счёта и применяет их по порядку.
- Часто для ускорения делают *snapshot*: периодически сохраняют готовое состояние, чтобы не проигрывать всю историю каждый раз.

В PHP есть библиотеки, облегчающие Event Sourcing, например **Broadway**, **EventSauce**. Они предоставляют инфраструктуру: хранилище событий (в БД или другом хранилище), механизмы воспроизведения и проекции.

Event Sourcing обычно идёт рука об руку с CQRS:

- Write-модель сохраняет события.
- Read-модель строится из событий (проекции). Например, события "ProductAddedToCart", "ProductRemovedFromCart" транслируются обработчиками в обновление таблицы "текущие корзины покупателей" для быстрого вывода на экран.

Преимущества Event Sourcing: полная история позволяет откатывать состояние, реконструировать любые моменты времени, аудит изменений "из коробки". Легко реализовать **compensating actions** (событие отмены) или проследить причины текущего состояния. **Недостатки:** сложность реализации, большой объём хранимых данных, необходимость проекций для чтения. Это оправдано в сложных доменах (финансы, бухгалтерия, где важна история) ([Domain-Driven Design with PHP | The PHP Consulting Company](#)) ([DDD, CQRS and Event Sourcing in PHP - GitHub Gist](#)).

Стоит упомянуть **реальные аналогии**: Event Sourcing похож на бухгалтерскую книгу – баланс счета не хранится напрямую, но вычисляется суммированием всех транзакций. Если где-то ошибка, можно пересчитать. В веб-разработке без Event Sourcing обычно сохраняется только текущее состояние (например, поле "баланс" в таблице), и некоторые данные истории теряются или хранятся отдельно (логи). ES объединяет состояние и лог изменений в единое хранилище.

Применять Event Sourcing следует осознанно. В PHP экосистеме есть успешные примеры: большие приложения на DDD + CQRS + ES, например, используя фреймворк **Prooph** или **Ecotone**. Но для небольших проектов ES излишне. Часто применяют гибридный подход: критичные агрегаты на ES, остальное – в обычной реляционной БД.

Связь с паттернами: Event Sourcing реализует паттерн *Observer* на уровне системы – все изменения превращаются в события, подписчики (проекции, другие контексты) на них реагируют. Также нужен паттерн *Repository*, который в данном случае сохраняет и загружает последовательность событий вместо данных напрямую.

3. Практические примеры паттернов: плюсы, минусы и области применения

Рассмотрим несколько практических примеров использования паттернов в PHP, с их преимуществами, недостатками и сценариями, где они применимы.

Пример 1: Стратегия (Strategy) для выбора способа оплаты

Ситуация: Допустим, мы разрабатываем модуль оплаты для интернет-магазина. Нам нужно поддерживать несколько платёжных шлюзов (например, кредитная карта, PayPal, криптовалюта). Новые способы оплаты могут добавляться со временем.

Проблема без паттерна: Можно было бы в классе Checkout сделать так:

```
class CheckoutController {
    public function pay(Order $order, string $method) {
        if ($method === 'card') {
            // логика оплаты картой
        } elseif ($method === 'paypal') {
            // логика оплаты PayPal
        } elseif ($method === 'crypto') {
            // логика оплаты крипто
        }
        // ...
    }
}
```

Этот код быстро разрастается по мере добавления вариантов. Он нарушает принципы SRP/ОСР: чтобы добавить новый метод оплаты, придётся менять класс, тестировать все ветки и т.д. Это **трудно поддерживать**.

Решение через паттерн Strategy: Выделим интерфейс стратегии оплаты и реализуем каждую логику в своём классе:

```
interface PaymentStrategy {
    public function pay(Order $order): bool;
}
```

```

class CardPayment implements PaymentStrategy {
    public function pay(Order $order): bool {
        // Логика оплаты кредитной картой
        return true;
    }
}

class PayPalPayment implements PaymentStrategy {
    public function pay(Order $order): bool {
        // Логика оплаты через PayPal API
        return true;
    }
}

class CryptoPayment implements PaymentStrategy {
    public function pay(Order $order): bool {
        // Логика оплаты криптовалютой
        return true;
    }
}

```

Теперь контроллер может работать с любой стратегией, не зная деталей:

```

class CheckoutController {
    public function pay(Order $order, PaymentStrategy $paymentMethod) {
        $success = $paymentMethod->pay($order);
        // ... дальнейшие действия
    }
}

```

При выборе способа (например, из UI) мы просто создаём нужный объект стратегии (new CardPayment() или др.) и передаем в pay(). Также можно использовать фабрику, возвращающую нужную стратегию по строковому коду.

Плюсы: Код **расширяем** – чтобы добавить новый метод оплаты, достаточно добавить новый класс, не трогая существующие (принцип *Open-Closed*). Логика каждого способа изолирована, её проще тестировать по отдельности. Можно даже конфигурировать стратегии в runtime (например, переключить провайдер платежей без перекомпиляции). Strategy хорошо подходит, когда алгоритмы могут подменяться взаимозаменяемо ([Design Patterns in PHP: Facade \(with Laravel\) - DEV Community](#)).

Минусы: Увеличивается количество классов. Для очень простых случаев это может показаться overkill. Однако для **сложно развивающихся систем** плюс перевешивает: паттерн убирает разрастание условных операторов.

Область применения: Когда есть несколько вариантов поведения, и выбор варианта может изменяться или расширяться. В вебе это всё, что поддерживает плагины, разные алгоритмы расчётов (скидки, налоги), разнообразные форматы вывода (например, стратегия сериализации JSON/XML). В Laravel, например, паттерн Strategy можно увидеть в механизме авторизации: Gate в Laravel поддерживает различные стратегии проверки (all, any) для политик доступа.

Пример 2: Наблюдатель (Observer) для рассылки уведомлений

Ситуация: Необходимо при регистрации нового пользователя выполнять несколько действий: отправить приветственное email-письмо, создать запись в CRM, уведомить администратора и т.д. Хотелось бы, чтобы код регистрации не "знал" обо всех этих побочных действиях, иначе он станет очень громоздким.

Решение через паттерн Observer (Издатель-Подписчик): Реализуем генерацию события "UserRegistered" и набор слушателей, которые будут реагировать.

В Laravel это можно сделать через встроенные **Events**: создать событие `UserRegistered` и слушатели `SendWelcomeEmail`, `AddToCRM` и т.п. При успешной регистрации просто вызвать `event(new UserRegistered($user));`. Laravel сам вызовет все подписанные слушатели (реализация observer pattern).

Если без фреймворка, можно сделать упрощённо:

```
class UserRegisteredEvent {
    public function __construct(public User $user) {}
}

class EventDispatcher {
    private array $listeners = [];
    public function subscribe(string $eventClass, callable $listener) {
        $this->listeners[$eventClass][] = $listener;
    }
    public function dispatch(object $event) {
        $eventClass = get_class($event);
        foreach ($this->listeners[$eventClass] ?? [] as $listener) {
            $listener($event);
        }
    }
}
```

Где-то в настройке системы подписываем слушателей:

```
$dispatcher->subscribe(UserRegisteredEvent::class, function(UserRegisteredEvent $e) {
    // Отправить email $e->user
});
$dispatcher->subscribe(UserRegisteredEvent::class, function(UserRegisteredEvent $e) {
    // Создать запись в CRM о $e->user
});
```

В коде регистрации:

```
// ... после сохранения пользователя:
$dispatcher->dispatch(new UserRegisteredEvent($user));
```

Плюсы: Код регистрации **изолирован** от побочных эффектов – он просто уведомляет, что "пользователь зарегистрирован". Добавить новое действие при регистрации можно, просто подписав нового слушателя, не трогая существующий код. Это снижает связность (low coupling) и улучшает масштабируемость приложения, когда со временем бизнес-правила обрастают новыми реакциями на события. Такой подход используется повсеместно: и Laravel Events, и Symfony EventDispatcher работают так же ([What is the difference between Repository Pattern and Facades ...](#)).

Минусы: Усложняется отслеживание потока выполнения – чтобы понять, что происходит при регистрации, нужно помнить про всех подписчиков. Неправильная обработка в слушателе может незаметно вызвать ошибку. Кроме того, если слушателей много, могут быть нюансы производительности (хотя обычно это незначительно). Для критичных процессов, требующих транзакционности, event- подход усложняет гарантии завершённости: нужно либо убедиться, что все слушатели успешно отработали, либо разруливать неуспех каждого по отдельности.

Область применения: Обработка событий, асинхронные действия, любая задача, где **несколько модулей должны реагировать на одно событие**. В фреймворках – отправка email, логирование действий, очистка кэша по событию обновления и т.д. Observer хорош, когда система должна быть **расширяемой** без изменения базового кода (например, плагиновая архитектура: плагин просто подписывается на событие).

(Другие примеры: можно было бы привести паттерн Decorator – например, декорирование логгера для добавления контекста, или паттерн Factory Method – например, фабрика создания разных типов уведомлений (SMS, Email) на основе конфигурации. Однако, в целях краткости, рассмотрены два вышеприведённых примера, демонстрирующих подход.)

4. Паттерны и тестируемость кода

Хорошо спроектированный код должен быть удобен для модульного тестирования. Правильное применение паттернов **повышает тестируемость** за счёт ослабления связности и выделения зависимостей. Однако некоторые паттерны, особенно если использовать их неправильно, могут затруднить тестирование.

Паттерны, улучшающие тестируемость:

- *Dependency Injection (DI)* – не зря многие называют DI паттерном, делающим код *тестопригодным*. Внедрение зависимостей означает, что класс не сам создаёт свои зависимости, а получает их извне (например, через конструктор) ([PHP and Dependency Injection: Decoupling and Testing PHP Applications | Reintech media](#)) ([PHP and Dependency Injection: Decoupling and Testing PHP Applications | Reintech media](#)). Это позволяет при тестировании подменить реальные зависимости на заглушки или моки. Например, класс, отправляющий email, может получать интерфейс Mailer – в тесте мы передадим ему MockMailer, и проверим, что вызван метод отправки, **без реальной отправки письма** ([PHP and Dependency Injection: Decoupling and Testing PHP Applications | Reintech media](#)) ([PHP and Dependency Injection: Decoupling and Testing PHP Applications | Reintech media](#)). Контейнер сервисов (Laravel, Symfony) облегчает управление DI, но сам по себе контейнер не обязателен для преимуществ DI.
- *Стратегия, Команда, Полиморфизм в целом*. Если логика реализована через общие интерфейсы и подставляемые реализации, то в тесте можно подставить специальную *тестовую реализацию*. Например, стратегию оплаты в тесте заменить на стратегию-имитацию, которая сразу возвращает успешный результат, чтобы проверить поведение остального кода. Или командный обработчик (Command Handler) можно протестировать изолированно, подменив репозиторий на InMemory-репозиторий.
- *Observer (Event)* – парадоксально, но событийную систему тоже можно упростить в тестах: вместо реального EventDispatcher передать фейковый, который просто собирает вызванные события в список – затем тест проверит, что нужное событие было отправлено. Laravel, кстати, имеет метод `Event::fake()` для этих целей.

Таким образом, сам факт применения observer-паттерна не мешает тестам, особенно если фреймворк поддерживает инструменты.

- *Factory* – фабрики могут помочь изолировать создание сложных объектов. В тесте фабрику можно настроить на создание упрощённых или предсказуемых экземпляров (например, всегда возвращать определённый объект). Также фабрику легко замочать, чтобы она возвращала подготовленный объект с известными данными.

В целом, паттерны, которые **разделяют ответственность и вводят абстракции**, обычно улучшают тестируемость. Принцип *низкого зацепления* из GRASP и SOLID принцип *Dependency Inversion* способствуют этому: модуль зависит от интерфейса, а не конкретной реализации – значит, в тесте можно подменить интерфейс.

Паттерны, ухудшающие тестируемость (или антипаттерны):

- *Singleton* – часто называется антипаттерном именно из-за тестирования ([Why is Singleton considered an anti-pattern? - Stack Overflow](#)) ([Why is Singleton considered an anti-pattern? - Stack Overflow](#)). Синглтон скрывает зависимости: класс внезапно внутри себя обращается к `Singleton::instance()` – в тесте сложно подменить такое поведение. Singletons ведут себя как глобальные переменные, которые трудно изолировать. Тесты должны запускаться в произвольном порядке, а глобальное состояние (которое не сбрасывается) этому мешает. Альтернатива: явное внедрение зависимости или использование контейнера, где можно зарегистрировать фейковую реализацию.
- *Service Locator* – по сути, тот же глобальный контейнер. Если класс сам лезет в глобальный контейнер за зависимостями, в тесте придётся подготавливать контейнер, регистрировать моки. Кроме того, Locator скрывает, что именно нужно классу (анализ кода сложнее) ([What makes the Service Locator pattern bad? : r/PHP](#)) ([What makes the Service Locator pattern bad? : r/PHP](#)). В результате, как отмечают разработчики, код с Service Locator зачастую плохо покрыт тестами ([What makes the Service Locator pattern bad? : r/PHP](#)) ([What makes the Service Locator pattern bad? : r/PHP](#)), потому что труднее писать. **Решение:** конструкторный DI, тогда тестировать проще.
- **Сильно связанный код без абстракций** – это скорее отсутствие паттерна (антипаттерн *спагетти-код*), но упомянуть стоит. Если у вас метод в 300 строк, который и читает БД, и пишет файл, и отправляет HTTP-запрос – чтобы протестировать его, нужно либо поднимать все окружение, либо рефакторить. Поэтому применение паттернов разделения обязанностей (MVC, слой услуг, репозитории и т.д.) – прямая выгода для тестирования. Например, вместо обращения к БД напрямую, класс получает интерфейс Repository – в тесте дадим ему фейковый репозиторий, и можем тестировать бизнес-логику без настоящей БД.
- *Паттерны с внутренней сложной схемой взаимодействия* – например, *Mediator*, *Observer* – могут усложнить написание тестов, так как нужно учитывать множественные объекты. Но этот минус не критичен: обычно достаточно писать модульные тесты на уровне выше (например, протестировать, что при вызове метода компонента-посредника результат достигается, не проверяя внутренние вызовы, или используя заглушки на участниках).

Итог: Большинство шаблонов при правильном использовании улучшают тестируемость, так как способствуют *де-композиции* и *инверсии зависимостей*. Главные враги тестов – *глобальное состояние* и *жёсткая связность*. Поэтому избегайте паттернов-антагонистов

(Singleton, Service Locator, Global Mediator без абстракций) либо компенсируйте их: например, Laravel-фасады статичны, но для тестов Laravel предлагает механизм *Mocking Facades* (замена под капотом сервисов). Опытный разработчик всегда думает: "А как я буду тестировать эту часть?". Если ответ сложный – возможно, выбран неверный дизайн.

5. Влияние паттернов на производительность

Паттерны проектирования влияют на структуру кода, а значит опосредованно могут влиять и на **производительность**. Чаще всего влияние это незначительное по сравнению с общей логикой, но в некоторых случаях выбор паттерна или архитектурного подхода может сказаться на скорости работы или потреблении ресурсов.

Абстракция vs Производительность: Использование паттернов обычно приводит к введению дополнительных уровней абстракции (больше методов, больше объектов). Каждый такой уровень – это дополнительные вызовы функций, иногда – дополнительные выделения памяти. В высокоуровневых языках (PHP) эти накладные расходы малы в контексте выполнения бизнес-логики или операций ввода-вывода. Однако в **критических местах** (например, в цикле, выполняющемся тысячи раз) лишние абстракции могут накопиться. Поэтому иногда ради оптимизации идут на нарушение чистоты паттерна (встроить что-то более напрямую).

Пример 1: ORM – Active Record vs Data Mapper. Интересно, что более “сложный” паттерн Data Mapper (Doctrine) иногда работает быстрее, чем Active Record (Eloquent) в массовых операциях. В одном исследовании сравнили выборку 1000 записей с несколькими связями: Eloquent (Active Record) vs Doctrine (Data Mapper). Результат: Doctrine на 10мс быстрее на запрос (39ms против 50ms) (~~Poor performance of Eloquent ORM in comparison to Doctrine | Kamil Ruczyński~~) (<https://sarvendev.com/2024/10/poor-performance-of-eloquent-orm-in-comparison-to-doctrine/#:~:text=Image%3A%20Eloquent%20hydration>) ([Poor performance of Eloquent ORM in comparison to Doctrine | Kamil Ruczyński](#)). Причина – Doctrine загружает связанные объекты через оптимизированный Unit of Work за один проход (после начальной гидрации данных объекты уже готовы для использования), а Eloquent делает дополнительную работу при доступе к связям (ленивая загрузка либо преобразования) ([Poor performance of Eloquent ORM in comparison to Doctrine | Kamil Ruczyński](#)). Это пример, как выбор паттерна архитектуры хранения данных влияет на производительность. Active Record проще, но может быть менее оптимален для сложных запросов, тогда как Data Mapper сложнее, но позволяет более гибкие оптимизации (кеширование, вычисление на стороне БД и т.п.).

Пример 2: DI-контейнер. Сервис-контейнер – реализация паттерна IoC – добавляет накладные расходы при разрешении зависимостей. Symfony компилирует контейнер в PHP-код, потому получение сервиса – это просто вызов функции (минимальная стоимость). Laravel генерирует контейнер на лету с рефлексией: на каждый запрос создания объекта тратится немного больше времени. В нагруженных сценариях (например, API, создающая сотни объектов на запрос) это заметно: было отмечено, что Laravel по умолчанию создает каждый сервис при каждом запросе заново (non-shared), что может быть неэффективно ([Uncovering the bottlenecks - An investigation into the poor performance of Laravel's container | Kamil Ruczyński](#)) ([Uncovering the bottlenecks - An investigation into the poor performance of Laravel's container | Kamil Ruczyński](#)). Решение – использовать *singleton* binding для часто используемых сервисов, чтобы контейнер кэшировал один экземпляр ([Uncovering the bottlenecks - An investigation into the poor performance of Laravel's container | Kamil Ruczyński](#)) ([Uncovering the bottlenecks - An investigation into the poor performance of Laravel's container | Kamil Ruczyński](#)). Symfony, как указано, по умолчанию *shareable* (всегда возвращает один экземпляр) ([Uncovering the](#)

[bottlenecks - An investigation into the poor performance of Laravel's container | Kamil Ruczyński](#)). Таким образом, сам по себе паттерн DI – благо для дизайна, но его наивная реализация (без учета кеширования) может стать узким местом. Оптимизация контейнера – задача фреймворка; разработчику важно помнить, что бесконтрольное создание объектов в циклах может тормозить, и при необходимости профилировать.

Паттерны, улучшающие производительность: Некоторые паттерны специально придуманы для оптимизации:

- *Lazy Loading (Virtual Proxy)* – отложенная инициализация. Позволяет не загружать объекты до тех пор, пока они не понадобятся. В Doctrine это ленивые прокси для связей: они экономят запросы к БД, загружая данные только при обращении.
- *Flyweight* – как уже сказано, экономит память при множестве повторяющихся объектов, разделяя общее состояние. В PHP это может быть, например, пул подключений или reuse крупных объектов вместо создания новых.
- *Caching Proxy / Decorator* – когда декоратор кеширует результат вызова следующего слоя. Например, обёртка над репозиторием, которая сначала смотрит в кеш (Redis), и только если промах – вызывает реальный репозиторий (БД). Этот шаблон существенно улучшает скорость повторных запросов ценой небольшой сложности.

Паттерны и сложность алгоритмов: Некоторые паттерны влияют на алгоритмическую сложность. Например, *Observer* с большим числом подписчиков – событие будет распространяться линейно по числу слушателей. *Chain of Responsibility* – худший случай, если каждое звено передаёт дальше, все N звеньев проверят запрос ($O(N)$). В реальности N редко бывает большим. Но если, к примеру, сделали цепочку из 1000 обработчиков, это плохо скажется на времени обработки.

Баланс архитектуры и оптимизации: В большинстве случаев следует сперва применять **паттерны для чистоты кода**, а затем профилировать узкие места. Как говорил известный афоризм, “преждевременная оптимизация – корень всех зол”. Однако **проектируя систему, важно учитывать масштаб**: паттерны вроде CQRS+ES дают потенциальный выигрыш производительности (отдельные read-модели, быстрые запросы по денормализованным данным), но цена – сложность. Если ожидается высокая нагрузка на чтение, целесообразно сразу заложить CQRS, иначе потом будет труднее вынимать логику на сторону чтения.

Пример 3: CQRS и производительность. Представим, у нас приложение с тысячами запросов в секунду, 90% из них – это чтение (отображение каталога товаров). Классическая архитектура (CRUD на одной модели) при росте нагрузок потребует тяжёлого горизонтального масштабирования БД, или ручного кеширования. Если же применён CQRS, можно вынести чтение на реплицированные базы или Elasticsearch. Тем самым производительность чтения масштабируется лучше. Минус – консистентность данных может быть **eventually consistent** (чтение чуть отстаёт), но для производительности веб-систем это часто приемлемо.

Вывод: Паттерны сами по себе – не панацея производительности. Любая дополнительная абстракция имеет небольшой оверхед, но грамотное использование паттернов позволяет писать код, который легче оптимизировать в будущем. Например, если у вас отделён слой репозитория, то внедрить в него кеширование (для ускорения) можно прозрачнее для остального кода. Если же SQL-запросы разбросаны повсюду, добавить кеш сложно. Таким образом, **паттерны создают возможности для оптимизации**. Иногда можно встретить утверждение, что паттерны “замедляют” программу – это справедливо только для тривиальных сценариев. На уровне enterprise-приложения правильная архитектура важнее, а узкие места решаются адресно: скажем, заменить некоторую часть на более эффективную реализацию (на C-расширение, или вынести в сервис).

Всегда полезно профилировать (Blackfire, Xdebug profiler) и видеть, где программа тратит время. Чаще всего bottleneck – обращение к БД, файлам, сетевые запросы, а не пару лишних вызовов методов. Тем не менее, зная внутренности фреймворков, **Senior-разработчик** может точно улучшить производительность: например, в Laravel отметить бины как singleton, использовать eager loading вместо N+1 запросов (что, по сути, выбор правильного паттерна загрузки данных), включить кеширование конфигурации (чтобы не парсить ее на каждый запрос). Это всё – применение паттернов (кеширование, ленивые вычисления) для ускорения.

6. Антипаттерны и альтернативные решения

Антипаттерны – это распространённые неэффективные решения, которых следует избегать. Зная их, проще распознать “запахи” в коде и применить правильные паттерны вместо них. Рассмотрим несколько популярных антипаттернов в PHP-разработке и как их преодолеть.

- **Божественный объект (God Object, "Big ball of mud")** – объект или класс, который делает **всё подряд**. Признаки: класс имеет огромное количество методов или огромный размер, множество ответственности (нарушение SRP) ([Anti-patterns that every developer should know - DEV Community](#)) ([Anti-patterns that every developer should know - DEV Community](#)). Пример: класс UserManager который и регистрацией занимается, и пароли шифрует, и письма шлёт, и отчёты генерирует. Такой код крайне трудно сопровождать и тестировать. **Решение:** декомпозировать. Разбить на несколько классов: UserService (только координирует), PasswordEncoder, EmailSender, UserRepository и т.д. То есть применить разделение ролей (GRASP Controller+Creator), использовать паттерны Facade (если нужен единый вход) или Mediator, чтобы координировать несколько небольших объектов. В итоге каждый компонент проще менять. Если нужен глобальный доступ к некоторым данным, вместо того чтобы пихать их в один класс, можно использовать паттерн Registry или DI-контейнер, но с осторожностью. Главное – **каждому классу свою роль**.
- **Спагетти-код** – неструктурированный код, где логика переплетена без чёткой архитектуры ([Anti-patterns that every developer should know - DEV Community](#)) ([Anti-patterns that every developer should know - DEV Community](#)). Обычно это скрипты с последовательностью операций, глобальные функции, переменные, обилие условных переходов (goto, break из нескольких уровней). Такой код трудно читать и изменять: любое изменение может повлиять на далёкие части. **Решение:** использовать архитектурные паттерны. В веб-приложениях MVC или слой сервисов разделяют логику представления, бизнес-правила и доступ к данным. Даже простой код можно структурировать: выделять функции с понятными именами, избегать глобальных переменных (использовать параметры и возвращаемые значения). Рефакторинг спагетти-кода начинается с "разрезания" на кусочки – если видите повторяющиеся фрагменты или блоки с комментарием "// какой-то этап", вынесите их в функцию или класс. Также важно внедрять **конвенции**: например, в Laravel-приложении следовать структуре каталогов (Models, Controllers, etc.) – это навязывает каркас, не дающий написать всё в routes.php. Советы Роберта Мартина в книге Clean Code также помогают бороться с "спагетти": короткие функции, понятные имена, удаление дублирования ([Anti-patterns that every developer should know - DEV Community](#)) ([Anti-patterns that every developer should know - DEV Community](#)). Кроме того, **рефакторинг** и покрытие тестами – союзники в поэтапном улучшении спагетти-кода.

- **Singleton Overuse (Злоупотребление одиночками)** – применение синглтонов повсюду вместо явных зависимостей. Если каждый второй класс получает данные через `Singleton::getInstance()`, система превращается в сплошные неявные зависимости. Это затрудняет тестирование (как мы упомянули), затрудняет параллелизм (если когда-нибудь понадобится, Singleton – глобальная точка сериализации). **Решение:** в большинстве случаев Singleton можно заменить паттерном *Singleton Registry* или DI-контейнером, где вы по-прежнему имеете один экземпляр, но он **внедряется явно**. Либо, если нужна глобальная точка доступа, использовать *статические методы* осторожно (Laravel фасады примерно так делают, но обеспечивают подмену). Главное – минимизировать области применения Singleton. Например, вместо `DBConnection::getInstance()` лучше передавать объект подключения в те классы, где он нужен, через конструктор. Тогда для тестов можно передать in-memory DB или mock.
- **Anemic Domain Model (Анемичная модель)** – с точки зрения DDD это тоже антипаттерн: классы-сущности, которые не содержат никакого поведения, а только данные (свойства и геттеры/сеттеры). Вся логика вынесена в сервисы, часто процедурные. По сути, нарушается принцип объектности: объект не инкапсулирует инварианты. **Признаки:** методы вида `get/set` и отсутствие бизнес-методов. **Проблема:** сложнее поддерживать консистентность, много дублирования (в сервисах проверяют данные, вместо того чтобы сам объект гарантировал правильное состояние). **Решение:** сместить логику внутрь сущностей, превратить *TransactionScript* в *Rich Domain Model*. Например, вместо сервиса `OrderService::addItem(Order $o, Product $p)` сделать метод `$order->addItem($product)`. Сервисы оставить только для координации контекстов или транзакций. Это приближает код к DDD.
- **Волшебная кнопка (Magic PushButton)** – термин, описывающий UI-обработчики, где по нажатию кнопки выполняется длинный неструктурированный сценарий (часто с множеством *copy-paste* кода). Часто встречается в старых PHP-проектах: один обработчик формы делает всё. **Решение:** как и спагетти, разнести по слоям, использовать MVC: контроллер обрабатывает запрос, валидирует, вызывает сервис(ы), которые инкапсулируют логику, и т.д. Вместо одного метода на 500 строк – 5 методов по 100.
- **Золотой молоток (Golden Hammer)** – не столько про код, сколько про подход: “любимым” паттерном пытаются решать все задачи. Например, разработчик везде пихает абстрактные фабрики, даже если достаточно простой функции; или слишком увлекается микро-сервисами, разделяя на сервисы то, что могло быть одним приложением (вне контекста). **Альтернатива:** всегда оценивать, подходит ли инструмент для задачи. Иметь в копилке много паттернов и подходов, и выбирать осознанно. Если ловите себя на том, что пишете громоздкую абстракцию для тривиальной вещи – шаг назад, вспомнить KISS (Keep It Simple). Иногда лучше простое решение, чем “супер-гибкое” переусложнённое.
- **Силовой конструктор (Heavy Constructor) / Теле-класс** – это когда объект загружает кучу зависимостей или данных в конструкторе, делая создание объекта дорогим и сложным. В PHP это может приводить к тому, что создание одного объекта тянет полприложения. **Решение:** ленивые загрузки, фабрики. Если объект действительно сложно создать (много параметров), используйте паттерн *Builder* или фабрику, чтобы не засорять конструктор логикой. Разбейте объект на более простые.

- **Антипаттерны проектирования БД:** использование незэкранированных SQL (SQL Injection), отсутствие нормализации (или избыточная нормализация), "божественная таблица", где всё в одном месте – это уже за гранью кода, но отражается. Для них альтернативы – правильное использование ORM или Query Builder, нормализация/денормализация по необходимости, и т.д.

Антипаттерны в Laravel/Symfony контексте:

- **Монолитный контроллер (Fat Controller)** – контроллер, нагруженный бизнес-логикой. Решение: вынос логики в модели (если Active Record) или сервисы/слои. В Symfony даже есть шутка "Thin Controller, Fat Model vs Fat Service".
- **Громоздкие модели (Fat Model)** – в Laravel модель может превратиться в God Object: помимо ORM-логики там и бизнес-правила, и валидация, и события. Решение: использовать подходы DDD – отделить доменные сервисы, value objects, и т.д., не класть всё в Eloquent модель. Можно внедрить *Repository* слой между контроллером и моделью, чтобы разгрузить модель.
- **Жёсткая привязка к фреймворку** – когда вся логика завязана на статических фасадах Laravel или контейнер Symfony. Это усложняет перенос и тестирование. Решение: изолировать core-логику от фреймворка насколько возможно (например, писать доменные классы, не знающие о Laravel, и тонкий слой, который их вызывает).

Под каждое зло обычно найдется паттерн-лекарство. Главное – распознать симптом. Например, видите 5 `if` с проверкой типа объекта – это кандидат на *Полиморфизм* или *Strategy*. Видите метод, возвращающий `true/false` и 10 мест, где на него подписываются действия – лучше сделать событие (*Observer*). Если в коде кусок, повторяющийся 3-4 раза с мелкими изменениями – подумать о *Template Method* или *Strategy*.

Заключение по антипаттернам: Не бойтесь признаваться, что текущий код – антипаттерн. Опытные разработчики ценны тем, что умеют рефакторить "плохой" код в "хороший". Используйте рефакторинг малыми шагами: покрыли тестами кусочек – заменили анти-шаблон на шаблон. Со временем кодовая база станет устойчивее и понятнее.

7. Популярные библиотеки, реализующие паттерны

PHP-экосистема богата библиотеками и фреймворками, которые либо сами являются реализациями паттернов, либо предоставляют готовые реализации паттернов для ваших нужд. Ниже – обзор некоторых популярных решений:

- **Laravel и Symfony (фреймворки)** – сами по себе демонстрируют множество паттернов. Laravel: *Active Record* (Eloquent), *Facade/Proxy* (фасады), *Decorator* (middleware), *Observer* (events, модель наблюдатель через Eloquent events), *Strategy* (разные драйверы кеша/очередей, выбираемые конфигурацией), *Builder* (Query Builder, Eloquent scopes). Symfony: *Front Controller* (HttpKernel), *MVC*, *Dependency Injection Container*, *Observer* (EventDispatcher), *Decorator* (HTTPPlug клиент-средний, или как Decorator in HttpKernel), *Data Mapper* (Doctrine ORM integration).
- **Doctrine** – набор проектов (ORM, DBAL, Migrations). Doctrine ORM – учебник по паттернам: *Unit of Work* (отслеживает изменения и применяет их к БД), *Data Mapper* (отделяет сущности от БД) ([My Digital Garden | Active Record vs. Data Mapper: Navigating PHP ORM Patterns with Eloquent and Doctrine](#)) ([My Digital Garden | Active Record vs. Data Mapper: Navigating PHP ORM Patterns with Eloquent and Doctrine](#)), *Identity Map* (хранит уже загруженные сущности, чтобы не

загружать дубликаты), *Repository* (наследуемый репозиторий для запросов). Doctrine DBAL (уровень ниже ORM) предоставляет *Builder* для запросов (QueryBuilder). Doctrine вдохновлена паттернами из книги Martin Fowler "Patterns of Enterprise Application Architecture".

- **PSR (PHP Standard Recommendations)** – это стандарты, но многие из них диктуют определённые паттерны: PSR-3 (Logger) – паттерн *Strategy* или *Observer* (множественные обработчики), PSR-6/16 (Cache) – *Adapter* (разные бекенды, один интерфейс), PSR-11 (Container) – *Service Locator/Registry*, PSR-14 (Event Dispatcher) – *Observer*. Реализации PSR обычно представлены библиотеками: например, **Monolog** (PSR-3) – реализует паттерн *Chain of Responsibility* для обработчиков логов (запись проходит через цепочку хэндлеров) и *Decorator* для форматтеров сообщений, **Symfony Cache** или **Laravel Cache** – реализуют *Adapter* (единый интерфейс над APCu, Redis, файлом и т.п.), **League Flysystem** – классический *Adapter* паттерн для файловых систем (локальная FS, S3, FTP – через единый интерфейс Filesystem).
- **Tactician** – библиотека Command Bus (применение паттерна *Command*). Она позволяет регистрировать обработчики команд и вызывать их через диспетчер. По сути, реализует инфраструктуру для CQRS (командную часть). Альтернатива – **SimpleBus** от Matthias Noback, или встроенный Symfony Messenger, который поддерживает и команды, и события.
- **Broadway** – фреймворк для Event Sourcing и CQRS от компании Qandidate. Предоставляет готовые компоненты: *Event Store*, *Read Model projector*, *Command handling*. Помогает реализовать DDD с Event Sourcing на PHP. Похожий современный – **EventSauce**, более легковесный подход к Event Sourcing, делающий акцент на простоте и вместо большого фреймворка предлагает небольшие building blocks.
- **Laravel Event Projector / Spatie Event Sourcing** – пакеты от Spatie для реализации Event Sourcing в Laravel. Они помогают настроить хранение событий (в таблице) и проекции read-моделей. Это отличные примеры *Observer* (слушатели проекций слушают события) и *Event Sourcing* практики.
- **Laravel Livewire / Symfony UX** – интересный паттерн *MVVM/Observer* для UI, где изменения на сервере автоматически обновляют UI. Livewire использует Observer-подход: компоненты отслеживают изменения состояний и генерируют обновления фронтенда.
- **PHP-DI** (внешний DI-контейнер) – реализует паттерн *Dependency Injection Container*, предоставляя альтернативу встроенному контейнеру Laravel или Symfony. Он позволяет определять правила, как создавать объекты, и инжектировать зависимости. Внутри PHP-DI тоже применяет *Reflection*, *Proxу* (для ленивых объектов), *Composite* (объединяя разные источники конфигурации).
- **Behat** – BDD-фреймворк, не совсем про паттерны, но внутри использует паттерн *Interpreter* (парсер Gherkin), *Observer* (события сценариев, на которые можно подписываться).
- **GuzzleHTTP** – HTTP-клиент. Он устроен по принципу *Middleware Stack* (цепочка middleware, напоминающая Chain of Responsibility, для обработки запросов/ответов) и *Facade* (класс `Client` фасадом предоставляет простые методы get/post вместо настройки запроса вручную).

- **GraphQL libraries** (например, `webonyx/graphql-php`) – используют *Builder* (для запросов схемы), *Interpreter* (разбор GraphQL запросов), *Visitor* (обход AST).
- **PHPUnit** – фреймворк тестирования – тоже пример: *Template Method* (методы `setUp/tearDown` вызываются по шаблону окружения тестов), *Composite* (сuits и тесты), *Command* (каждый тест как команда, которую runner выполняет), *Observer* (listeners на события тестового прогона).
- **Design Patterns PHP** – отдельно стоит упомянуть репозиторий **DesignPatternsPHP** ([2.6. Dependency Injection – DesignPatternsPHP 1.0 documentation](#)) ([2.6. Dependency Injection – DesignPatternsPHP 1.0 documentation](#)) (на GitHub) – это скорее учебный проект, демонстрирующий реализацию всех GoF-паттернов на PHP с примерами. Он не для продакшена, но как справочник отлично показывает, как реализовать тот или иной паттерн на PHP 8, и содержит полезные комментарии.
- **Refactoring.Guru** – это не библиотека, а ресурс, но у них есть раздел с примерами паттернов на PHP. К тому же сайт доступен на русском. Он может служить справочником, хотя не библиотекой.

Внедрение сторонних библиотек: В экосистеме PHP зачастую лучше взять готовую реализацию паттерна, чем писать свою – это экономит время и, вероятно, уже хорошо протестировано. Например, вместо писать свой Event Dispatcher с нуля – можно подключить `symfony/event-dispatcher` (легко интегрируется и вне Symfony, реализует PSR-14). Вместо своей логгерной системы – `monolog/monolog`. Это пример повторного использования паттернов через библиотеки.

Важно понимать, что каждая библиотека – инструмент. Senior-разработчик оценивает, стоит ли притягивать зависимость или достаточно своего простого решения. Например, если нужно один раз вызвать команду, можно и без шины команд обойтись, вызвав сервис. Но если архитектура идёт к полноценному CQRS – лучше сразу ввести Tactician или Messenger.

Наконец, **Composer пакеты** часто отражают паттерны в названиях: "adapter", "bridge", "middleware", "manager" – при выборе пакета можно догадаться о заложенном паттерне. Например, *HTTPPlug* – абстракция (Adapter) над HTTP-клиентами; *The PHP League's Fractal* – реализует паттерн *Transformer* (преобразование моделей в структуры для API – часто относят к Presentation паттернам).

8. Объяснение сложных концепций на простых примерах

Иногда сухое описание паттерна недостаточно для понимания – помогает аналогия с реальной жизнью или упрощённый пример.

- **Фасад (Facade):** *Аналогия:* Представьте, что вы пришли в ресторан. Вместо того чтобы общаться с поварами, сомелье, официантами отдельно, вы общаетесь только с метрдотелем. Метрдотель – фасад: он принимает вашу просьбу "организуйте ужин", а внутри координирует работу кухни, бара и официантов. Для вас интерфейс – один человек, скрывающий сложность командной работы. Так же и в коде: объект-фасад предоставляет простой метод, внутри которого вызывает множество методов других классов ([Does Facade in Laravel implements Facade or Proxy Pattern? - Stack Overflow](#)) ([Does Facade in Laravel implements Facade or Proxy Pattern? - Stack Overflow](#)). В Laravel фасадом можно считать, например, `Route` фасад – вы вызываете `Route::get(...)`, а под капотом Laravel настроит и контроллер, и привязки, что угодно.

- **Стратегия (Strategy):** *Аналогия:* Вы едете из точки А в точку В. Можно на машине, можно на велосипеде, можно пешком. Способ – стратегия. У всех одна цель – достичь В, но алгоритмы (маршрут, скорость) разные. Вы выбираете стратегию по условиям: далеко – машина, близко – пешком, хорошая погода – велосипед. В коде мы аналогично подставляем алгоритм по ситуации. В интернет-магазине стратегия выбора способа оплаты – из той же оперы.
- **Наблюдатель (Observer):** *Аналогия:* Подписка на журналы или на канал в YouTube. Есть издатель (журнал, автор канала), есть множество подписчиков. Когда выходит новый выпуск, издатель рассылает его всем подписчикам. В коде издатель (Subject) вызывает метод уведомления, и у каждого подписчика (Observer) срабатывает свой обработчик ([What is the difference between Repository Pattern and Facades ...](#)). Это позволяет издателю не задумываться, что конкретно делают получатели – каждый сам решает. Примером в жизни также служит сигнализация: датчик дыма (субъект) при срабатывании оповещает пожарную систему и, скажем, sms-уведомление владельцу (два наблюдателя на одно событие).
- **Декоратор (Decorator):** *Аналогия:* Возьмём новогоднюю ёлку. Сама по себе ёлка – объект (базовый компонент). Мы можем **декорировать** её: повесить гирлянду, шары, мишуру. Каждый декор – доп. функциональность, которая оборачивает ёлку, но сама ёлка при этом остаётся ёлкой и может использоваться как раньше. В программировании: есть базовый объект (например, поток данных), и декораторы: шифрование поверх потока, сжатие, буферизация. Каждый декоратор добавляет поведение (encrypt, compress), но поддерживает тот же интерфейс чтения/записи. Можно накладывать несколько декораторов как слоёв обёртки.
- **MVC (Model-View-Controller):** *Аналогия:* Онлайн-заказ пиццы. **Model** – сама пицца (состав, данные о заказе), **View** – веб-страница или приложение, на котором вы видите меню и заполняете заказ (представление), **Controller** – оператор call-центра или система приема заказов, которая получает ваш запрос ("хочу пиццу с такими-то ингредиентами") и запускает процесс (передает на кухню и т.д.). В приложении Controller принимает HTTP-запрос, дергает Model (например, сохранить заказ) и возвращает View (страницу подтверждения). Разделение гарантирует, что логика приготовления пиццы (Model: рецепты, ингредиенты) отделена от того, как выглядит меню (View), и от того, как обрабатывается ваш звонок (Controller).
- **Repository:** *Аналогия:* Библиотека. У вас есть библиотекарь (Repository), к которому вы обращаетесь: "дай книгу с таким-то названием". Библиотекарь знает, где лежит эта книга (в каком фонде, на каком стеллаже), приносит её вам. Вы не ходите сами по хранилищу – для вас интерфейс упрощён (по имени книги). Так и Repository в коде – предоставляет методы, например, `findById`, `findByName`, а внутри может делать сложный запрос к БД или даже к нескольким. Domain-слою все равно, как именно данные получены – он полагается на "библиотекаря".
- **Unit of Work:** *Аналогия:* Секретарь, собирающий все правки документов за день, а затем разом обновляющий архив. Вместо того, чтобы сразу при каждом изменении бегать в архив (что долго), секретарь помечает: документ А изменить, В удалить, С добавить. В конце дня применяет все изменения пачкой. Unit of Work (единица работы) – паттерн, который в ORM копит изменения объектов и затем одним транзакционным блоком сохраняет их. Это повышает эффективность (меньше запросов) и целостность (все изменения применились или ни одно). Doctrine ORM как раз это делает: после изменений сущностей вызываем `flush()`, и UoW сам решает, какие SQL выполнять.

- **CQRS:** *Пример из жизни:* Представьте, есть склад и магазин. В магазине вам показывают витрину с товарами – это быстрый способ узнать, что в наличии (денормализованное представление). Но когда вы покупаете товар, склад списывает его со своих учётов – это отдельный процесс. Магазин (витрина) время от времени обновляется данными со склада. Здесь склад – Write-модель (единственный источник истины), витрина – Read-модель (отдельная, оптимизированная под чтение, может немного отставать). Так и CQRS: командная модель строго ведёт учёт, а запросная – быстро отдаёт информацию, возможно, через копию данных. Другой пример – отчёты и OLTP: транзакционная БД записывает все операции (commands), а отчётная система (data warehouse) периодически загружает их для быстрых запросов (queries).
- **Event Sourcing:** *Аналогия:* Банковский счёт – классический пример. Баланс счёта – это не просто число, это результат последовательности операций (событий): зачисление зарплаты, снятие оплаты за аренду, покупка в магазине... Если хранить только баланс, вы потеряете историю операций. В Event Sourcing вы храните *все операции*. Баланс можно посчитать, просуммировав события. Если ошибка, можно отменить (внести противоположное событие) или пересчитать. Это похоже на журнал транзакций в бухгалтерии или ленту событий в истории пользователя (например, история правок на Wikipedia – факты изменений). С помощью полного журнала можно воспроизвести состояние на любую дату – это мощное преимущество.
- **Dependency Injection:** *Аналогия:* Представьте электрический прибор и розетку. Прибору нужен ток (зависимость). Один вариант – прибор сам генерирует электричество (встроенная батарея). Другой – **внедрение зависимости**: прибор ничего не знает о производстве электричества, он имеет разъём, через который подключается к розетке. Розетка подаёт ток извне. В программировании DI – это когда объект получает необходимые ресурсы извне (через конструктор, сеттер или контекст) ([2.6. Dependency Injection – DesignPatternsPHP 1.0 documentation](#)) ([2.6. Dependency Injection – DesignPatternsPHP 1.0 documentation](#)), а не создает их сам. Например, если класс контроллера сам внутри делает `$db = new DatabaseConnection()`, то он "сам себе батарейка". Если же мы передаем `DatabaseConnection` в конструктор контроллера – мы подключили его к "розетке". Это позволяет подменить "розетку" при тестировании (например, дать ему фейковую БД) или переключить источник (например, вместо MySQL дать MongoDB, поменяв только конфигурацию DI).
- **Полиморфизм (и принцип подстановки LSP):** *Пример:* У вас есть пульт дистанционного управления (интерфейс) и разные модели телевизоров (реализации). Пульт не знает внутренностей ТВ, но при нажатии "Вкл" любой телевизор включится, если он понимает этот интерфейс (у всех есть метод `turnOn()`). Вы можете заменить телевизор на другой – пульт работать продолжит. В коде это значит, что вместо конкретных классов мы кодируем против интерфейсов. Например, мы можем иметь интерфейс `LoggerInterface` и передавать либо `Monolog`, либо свой класс – потребителю без разницы, он вызывает `$logger->log()`.
- **SOLID принципы** тоже легко пояснить:
 - SRP (Single Responsibility) – *одно класс – одна ответственность*. Как прибор: стиральная машина стирает, духовка печет, не надо делать "стиральнопылесосомикроволновку".
 - OCP (Open-Closed) – "открыт для расширения, закрыт для изменения": автомобиль открыт для апгрейда (можно поставить другой аккумулятор), но

при этом базовая конструкция закрыта от изменения (вам не нужно перепаявать проводку). В коде – можно добавить новый класс-наследник, но не менять базовый код.

- LSP (Liskov Substitution) – наследник должен полностью заменять родителя. Если у вас вместо обычной лампочки поставить "умную лампочку", она должна хотя бы светить как обычная, и не взрываться от простого выключателя.
- ISP (Interface Segregation) – многие специфичные интерфейсы лучше, чем один жирный. Пример: в ресторане шеф-повар не обязан быть одновременно и официантом, и бухгалтером. Пусть будет три человека на три задачи, чем один, который умеет всё, но занят чрезмерно.
- DIP (Dependency Inversion) – "зависеть от абстракций, не от деталей": вы звоните по телефону, не важно, айфон у собеседника или самсунг – важно, что оба поддерживают протокол связи. В коде – принимать интерфейс `SmsSender`, а конкретную реализацию (модем или API Twilio) подставить извне.

Такие аналогии помогают быстро понять суть. Senior-разработчик может объяснить коллегам: "Здесь у нас будет фасад, представьте, как фронт-офис в банке", или "Давайте не делать god-object, в реальной жизни же у вас не один робот на весь завод выполняет все операции!". Уместные метафоры облегчают коммуникацию в команде.

9. Шпаргалка по паттернам (краткое резюме)

Ниже представлена структурированная шпаргалка, описывающая основные паттерны и концепции, о которых шла речь, с кратким пояснением их назначения:

Порождающие паттерны (Creational):

- **Factory Method (Фабричный метод)** – делегирует создание объекта подклассам. *Когда использовать:* нужно создавать семейство объектов через общий интерфейс, но выбор класса зависит от контекста.
- **Abstract Factory (Абстрактная фабрика)** – интерфейс для создания множества взаимосвязанных объектов. *Применение:* UI-библиотеки (разные темы виджетов), переключение наборов драйверов.
- **Builder (Строитель)** – поэтапно конструирует сложный объект, отделяя процесс построения от представления. *Пример:* сборка XML/SQL через шаги (как Query Builder ([08 - Builder Pattern: You're Already Using It Daily | Laravel Daily](#)) ([08 - Builder Pattern: You're Already Using It Daily | Laravel Daily](#))).
- **Prototype (Прототип)** – создание нового объекта путем клонирования прототипа. *Использование:* когда создание с нуля дорого, а у вас есть подобный объект (например, клонировать конфигурацию и слегка поменять).
- **Singleton (Одиночка)** – один экземпляр класса в приложении. *Использование:* редко оправдан, пример – класс конфигурации или connection pool. Но помните о плюсах/минусах (может усложнить тесты).

Структурные паттерны (Structural):

- **Adapter (Адаптер)** – приводит интерфейс объекта в вид, нужный клиенту. *Например:* адаптер для платежного шлюза, чтобы наш код вызывал методы `charge()` независимо от поставщика.
- **Bridge (Мост)** – разделяет абстракцию и реализацию в разные иерархии, соединяя их через композицию. *Когда:* нужно менять реализацию независимо от абстракции (напр. разные движки рендеринга для одного API рисования).

- **Composite (Компоновщик)** – композиция объектов в древовидную структуру для работы с группой так же, как с отдельным объектом. *Пример:* меню с подпунктами, файловая система (папка содержит файлы/папки, но и то и другое – "узлы").
- **Decorator (Декоратор)** – динамически добавляет функциональность объекту, оборачивая его. *Пример:* добавление логирования к сервису путем обёртки, не меняя исходный сервис; оформление HTML-вывода.
- **Facade (Фасад)** – единый упрощённый интерфейс к сложной подсистеме. *Пример:* класс, инкапсулирующий работу с несколькими модулями (вызов одного метода вместо множества настроек) ([Does Facade in Laravel implements Facade or Proxy Pattern? - Stack Overflow](#)) ([Does Facade in Laravel implements Facade or Proxy Pattern? - Stack Overflow](#)).
- **Flyweight (Приспособленец)** – разделение состояния объектов для эффективного использования памяти. *Пример:* объект символа в текстовом редакторе: общий объект на каждую букву "А" со свойствами, различающимися только позицией.
- **Proxy (Прокси)** – суррогат, контролирующий доступ к реальному объекту. *Варианты:* удалённый прокси (для распределённых систем), виртуальный прокси (ленивая загрузка), защищающий прокси (контроль доступа). *Пример:* Laravel Facade – статический прокси к объекту в контейнере ([Facades - Laravel 11.x - The PHP Framework For Web Artisans](#)).

Поведенческие паттерны (Behavioral):

- **Chain of Responsibility (Цепочка обязанностей)** – передача запроса по цепочке обработчиков. *Пример:* HTTP middleware, фильтры событий; первый подходящий обработчик выполняет действие ([Drawbacks behind Laravel Facades - Codito.dev](#)).
- **Command (Команда)** – объект-запрос, позволяющий логировать, очереждать и отменять операции. *Применение:* очереди задач (jobs), команды в CQRS.
- **Interpreter (Интерпретатор)** – для заданного языка определяет представление его грамматики и интерпретирует высказывания. *Пример:* разбор простых скриптов или формул; в PHP мало используется, разве что в парсерах или шаблонизаторах.
- **Iterator (Итератор)** – последовательный доступ к элементам коллекции без раскрытия внутреннего представления. *В PHP:* `IteratorAggregate`, `yield` и т.п. Итератор – часто неявно, встроен в язык.
- **Mediator (Посредник)** – определяет объект, который централизует коммуникацию между другими. *Пример:* чат-рум, диспетчер диалоговых окон (каждое окно не знает о другом, общение через посредника).
- **Memento (Хранитель)** – сохраняет снимок внутреннего состояния, чтобы потом восстановить. *Пример:* система отката (undo) в приложении, сохранение игры.
- **Observer (Наблюдатель, Издатель-Подписчик)** – объект (Subject) рассылает уведомления множеству наблюдателей при изменении состояния ([What is the difference between Repository Pattern and Facades ...](#)). *Пример:* событийная система (подписка на события), нотификации (Laravel Events, Symfony EventDispatcher).
- **State (Состояние)** – объект меняет свое поведение при смене состояния, как будто изменился его класс. *Пример:* объект заказа с под-объектами состояния "Новый", "Отменён", "Отгружен" – методы `cancel()` или `ship()` делегируют на текущее состояние.
- **Strategy (Стратегия)** – семейство взаимозаменяемых алгоритмов, инкапсулированных в отдельные классы ([Design Patterns in PHP: Facade \(with Laravel\) - DEV Community](#)). *Пример:* алгоритмы сортировки, способы оплаты, методы расчёта цены.
- **Template Method (Шаблонный метод)** – в базовом классе определен общий алгоритм с вызовами шагов, которые реализуются в подклассах. *Пример:* алгоритм экспорта, где разные форматы меняют только часть процесса.

- **Visitor (Посетитель)** – отделение операции от объектов, над которыми она выполняется. *Пример:* обход разных узлов AST с выполнением операции, не засоряя сами классы узлов множеством методов.

Принципы и подходы:

- **SOLID** – 5 принципов: Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, Dependency Inversion. Помогают направлять дизайн (см. пояснения аналогий выше).
- **GRASP** – 9 принципов распределения обязанностей (Информационный эксперт, Создатель, Контроллер, Слабое сцепление, Высокая связанность, Полиморфизм, Индикация, Чистая выдумка, Защищенные вариации) – применяются при высокоуровневом проектировании классов.
- **DRY (Don't Repeat Yourself)** – не дублируй код; часто достигается через вынос повторяющейся логики в общий модуль или применение паттерна вместо копипаста.
- **KISS (Keep It Simple, Stupid)** – не усложняй без необходимости; паттерн ради паттерна – плохая практика. Лучше простое решение, если хватает, чем сложное сверх меры (избегать "golden hammer").
- **YAGNI (You Aren't Gonna Need It)** – не закладывай функциональность, которая не нужна сейчас. В контексте паттернов: не надо имплементировать 10 стратегий, если приложение поддерживает одну, но **будь готов** расширить, когда понадобится.

Современные архитектурные паттерны:

- **Domain-Driven Design (DDD)** – методология, где дизайн системы строится от бизнес-домена. *Основные паттерны:* Entity, Value Object, Aggregate Root, Repository, Domain Service, Factory, Domain Event. Цель – связать код с реальностью бизнеса, изолировать доменные модели, обеспечить понятность и гибкость в сложных проектах.
- **CQRS** – отдельные модели/слои для команд (изменений) и запросов (чтения). *Следствие:* можно отдельно масштабировать чтение, оптимизировать запросы, хранить проекции.
- **Event Sourcing** – хранение событий вместо состояния. *Плюсы:* полная история, легко сочетать с CQRS (события -> проекции). *Минусы:* требует продуманной реализации, не всегда оправдан в простых приложениях.
- **Clean/Hexagonal Architecture** – многослойная архитектура (доменный слой в центре, интерфейсы к инфраструктуре на периферии). *Ports and Adapters:* абстракции для взаимодействия с внешним миром (БД, UI). Это не конкретный паттерн, а подход организации, часто сочетается с DDD.
- **Microservices** – выделение отдельных сервисов со своим контекстом. *Связь с паттернами:* применяются те же DDD-контексты (Bounded Context), интеграция через события (Event-Driven Architecture), API Gateway (как Facade для набора сервисов).

Антипаттерны:

- **God Object** – класс, делающий всё; решение: декомпозиция, SRP.
- **Spaghetti Code** – хаотичный код без структуры; решение: рефакторинг в MVC/слои, упорядочение.
- **Singleton Abuse** – чрезмерное использование синглтонов; решение: DI контейнер, явные зависимости.
- **Anemic Model** – "бледная" модель без логики; решение: перенести логику в объекты (Rich Model).
- **Golden Hammer** – применение одной техники ко всем проблемам; решение: быть гибким, изучать разные паттерны.

- **Copy-Paste Programming** – дублирование кода вместо абстрагирования; решение: вынести общее в функции/классы, использовать шаблонные методы или стратегии там, где ветви кода похожи.
- **Early Optimization** – чрезмерная оптимизация в ущерб читаемости; решение: придерживаться KISS, оптимизировать после профилирования реальных узких мест.

Заключение: Понимание и грамотное применение паттернов – признак senior-разработчика. Этот гайд охватил классические шаблоны (GoF, GRASP) с примерами на PHP 8+, современные архитектурные подходы (DDD, CQRS, Event Sourcing), а также рассмотрел влияние решений на тестируемость и производительность, типичные ошибки проектирования и их решения. Пользуясь шпаргалкой, вы быстро освежите в памяти ключевые паттерны и принципы. Но важно помнить: паттерны – это инструмент, а не цель. Используйте их с умом, руководствуясь потребностями проекта, и ваш код будет одновременно элегантным, поддерживаемым и эффективным.